

QuectoC Phase 2

Standalone project setup guide

MountainC / Introductory preview

Create and run a small syntax checker using **Flex**, **Bison** and **GCC**. This guide shows the project tree, installations, complete core source files, build steps and exactly which files change at each checkpoint.

Companion to **MountainC_Phase_2_QuectoC_Introductory_Preview.pdf**. The standalone project starts at checkpoint 1 and requires no earlier Phase 1 repository.

Checkpoint	Syntax accepted	Main change
1 - One declaration	<code>let total=12;</code>	Create the scanner, grammar, driver and build script.
2 - Expressions	<code>let total=(12+3)-1;</code>	Extend parser.y ; update expected test results.
3 - Statements	<code>let total=12;</code> <code>print(total);</code>	Extend parser.y again; update expected results.

How to use this guide

Set up the terminal on page 2, unpack the project on page 3, then build and test using pages 8-10. Pages 5-8 explain every core file. Complete the grammar changes on pages 11-12. Record evidence on page 14; use pages 15-16 for the optional CMake build.

Included companion: MountainC_Phase_2_QuectoC_Standalone_Project.zip contains the guide, a ready-to-build checkpoint 1 project, all three reference checkpoints, 26 test inputs per checkpoint and a reporting template.

Scope: syntax checking only. This program does not evaluate expressions, execute print, build an AST or check declarations and types. The recognized keyword `int` has no grammar rule in this preview.

Prepare the Linux terminal

Route A - GitHub Codespaces from a Windows browser

Create a new GitHub repository, for example **quectoc-phase2-preview**, and initialize it with a README so its branch is not empty. Upload the companion ZIP to that repository using GitHub's file upload action. Open **Code > Codespaces** and create a codespace on the selected branch. Use its integrated terminal for the commands below. [3]

The browser is on Windows; these commands run in the Linux codespace. Install the tools inside that terminal.

Route B - Ubuntu lab machine or existing Ubuntu VM

Download the companion ZIP, place it in a working directory you can write to, and open a terminal there. Use a code editor to edit the files. The build runs in the terminal.

Install the required tools

```
sudo apt update
sudo apt install -y flex bison build-essential python3 git unzip tree
bison --version
flex --version
gcc --version
python3 --version
git --version
```

Tool	Role in this project
Bison 3.6 or newer	Converts parser.y into the parser and shared token header.
Flex	Converts scanner.l into the C scanner.
build-essential / GCC	Provides the C compiler and supporting build tools.
Python 3	Runs tests/check.py; it is not part of the parser executable.
unzip / tree / Git	Unpack the project, display its folders and record source changes.

Install once per environment. If a codespace is rebuilt or replaced, check the versions again and reinstall missing tools. The guide uses Bash and the supplied build.sh script.

Unpack and inspect the project

Run these commands from the folder containing the companion ZIP. They unpack a new standalone folder. If a folder with this name already exists, keep your edited copy and choose a different extraction location.

```
unzip MountainC_Phase_2_QuectoC_Standalone_Project.zip
cd quectoc-phase2-preview
pwd
ls
tree -a -L 2
```

Project root means the directory containing scanner.l, parser.y, main.c and build.sh. All following build and run commands start here.

quectoc-phase2-preview/

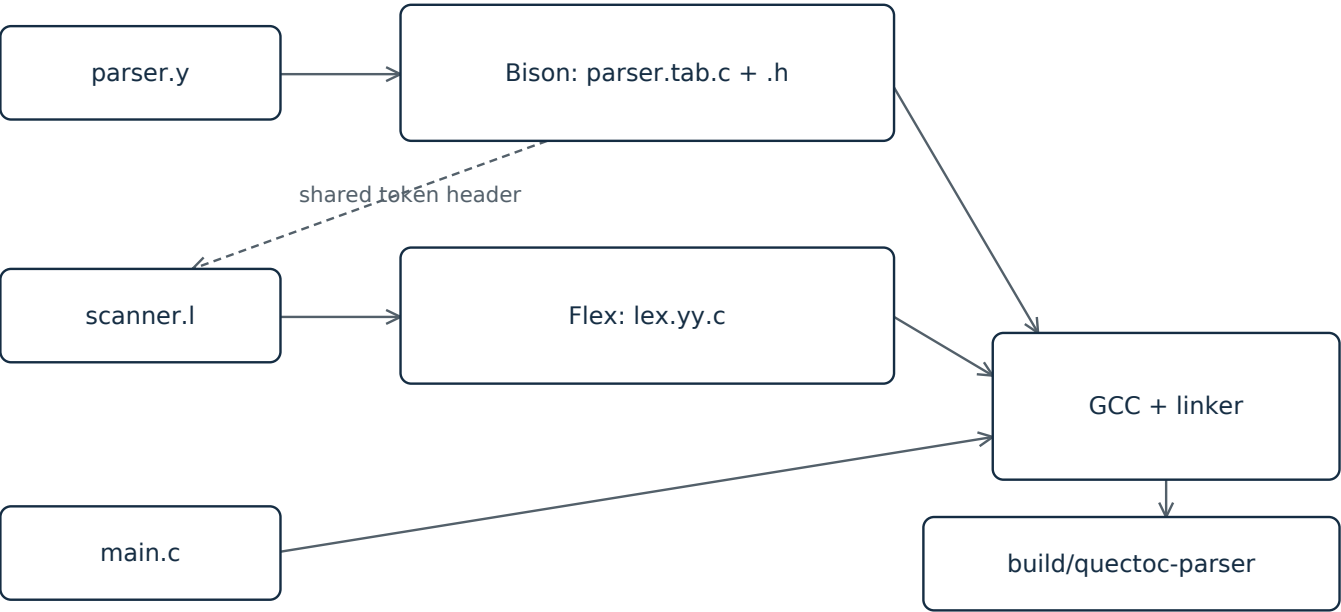
```
— scanner.l
— parser.y
— main.c
— build.sh
— CMakeLists.txt
— .gitignore
— README.md
— Validation.txt
— tests/
— Reporting/
  |— notes.md
— Checkpoints/
  |— 01_One_Declaration/
  |— 02_Expressions/
  |— 03_Statements/
— docs/
— build/
— build-cmake/
```

project root

```
Flex source; all checkpoints
Bison grammar; edit at each checkpoint
C driver; the only main()
generation, compilation and linking
optional command-line CMake build
ignore generated build output
quick-start instructions
verification record
26 .qc inputs + cases.json + check.py
student learning evidence
complete this template
reference copies
complete checkpoint 1
complete checkpoint 2
complete checkpoint 3
this setup guide
created by build.sh
created only when using CMake
```

Edit the top-level working files. Keep Checkpoints/ as a reference. Both build folders are initially absent. Each reference checkpoint is independently runnable with its own build.sh and tests.

How the files connect



Bison generates **build/parser.tab.c** and **build/parser.tab.h**. Flex generates **build/lex.yy.c**. GCC compiles the two generated C files and main.c, then links them into one executable. The dashed dependency shows that the scanner source includes Bison's token header. [1, 2]

Stage	What happens	File to edit
Build time	Recognize patterns and grammar rules, then generate and compile C code.	scanner.l, parser.y, main.c, build.sh
Run time	main() calls yyparse(); yyparse() calls yylex() whenever it needs another token.	Use the executable with a .qc input.
Scanner response	Return one token kind; skip white space and comments; return 0 only at end of input.	scanner.l
Parser response	Check the sequence against program and its grammar rules.	parser.y
Final report	main.c prints Syntax accepted. only if parsing succeeds and no lexical error was recorded.	main.c

Example token sequence: let total=12; becomes KW_LET, IDENTIFIER, ASSIGN, INT_LITERAL, SEMICOLON, followed by end of input. Token codes come from parser.tab.h. Their numeric values should not be written by hand.

Create scanner.l

Location: quectoc-phase2-preview/scanner.l. This complete file is already supplied. It remains the same for all three checkpoints.

```
%option noyywrap noinput nounput yylineno never-interactive nodefault
%{
#include <stdio.h>
#include "parser.tab.h"
/* A lexical error must prevent an overall success message. */
int lexical_error = 0;
%}
%%
"let"          { return KW_LET; }
"int"          { return KW_INT; }
"print"        { return KW_PRINT; }
[A-Za-z_][A-Za-z0-9_]* { return IDENTIFIER; }
[0-9]+         { return INT_LITERAL; }
"="           { return ASSIGN; }
"+"          { return PLUS; }
"- "         { return MINUS; }
";"          { return SEMICOLON; }
"("          { return LPAREN; }
")"          { return RPAREN; }
"//[^\n]*     { /* Skip a line comment, even at EOF. */ }
[ \t\r\n]+   { /* Keep scanning across newlines. */ }
<<EOF>>      { return 0; }
. {
    fprintf(stderr, "Lexical error at line %d: unexpected '%s'.\n",
              yylineno, yytext);
    lexical_error = 1;
    return LEX_ERROR;
}
%%
```

Why these actions matter: return KW_LET sends a token to the parser. A skip action keeps scanning without returning a token. The next call to yylex() resumes at the current input position. The fallback records a lexical error and returns LEX_ERROR. [1]

Keyword rules precede the identifier rule. There is no standalone main() or emit() helper in this file. The two %% lines separate definitions, rules and the optional final C-code section.

Create parser.y - checkpoint 1

Location: quectoc-phase2-preview/parser.y. Start with this complete file. It accepts exactly one declaration, followed by end of input.

```
%require "3.6"
%{
int yylex(void);
void yyerror(const char *message);
%}
%define parse.error detailed
%define parse.lac full
%token KW_LET "let" KW_INT "int" KW_PRINT "print"
%token IDENTIFIER "identifier" INT_LITERAL "integer"
%token ASSIGN "=" PLUS "+" MINUS "-" SEMICOLON ";"
%token LPAREN "(" RPAREN ")" LEX_ERROR "invalid character"
%start program
%%
program:
    KW_LET IDENTIFIER ASSIGN INT_LITERAL SEMICOLON
;
%%
```

Element	Purpose
%require "3.6"	Reject an older Bison installation.
%token declarations	Declare every token name used by the scanner. Bison puts their codes in parser.tab.h.
%start program	Choose the grammar's start symbol.
program: ... ;	Require let, an identifier, =, an integer and a semicolon in this order.
SEMICOLON versus ;	SEMICOLON represents the source-language token. The final unquoted ; ends the Bison rule.
parse.error / parse.lac	Request detailed syntax errors with lookahead correction.

For later checkpoints, keep everything above the first %% line and keep the closing %% line. Change the grammar between those markers. Do not create a second parser.y in tests/ or build/.

Boundary: int total=12; is rejected even though the scanner recognizes int. A recognized token does not automatically have a valid place in the grammar.

Create main.c - the driver

Location: quectoc-phase2-preview/main.c. This is the only program entry point. It supports either a filename argument or standard input.

```
#include <stdio.h>
#include "parser.tab.h"

extern FILE *yyin;
extern int yylineno;
extern int lexical_error;
int yylex_destroy(void);

void yyerror(const char *message)
{
    /* The scanner has already explained a lexical failure. */
    if (!lexical_error)
        fprintf(stderr, "Syntax error at line %d: %s\n", yylineno, message);
}

int main(int argc, char **argv)
{
    if (argc > 2) {
        fprintf(stderr, "Usage: %s [source.qc]\n", argv[0]);
        return 2;
    }
    yyin = stdin;
    if (argc == 2) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror(argv[1]);
            return 2;
        }
    }
    /* The parser requests tokens from the scanner as needed. */
    int status = yyparse();
    if (yyin != stdin)
        fclose(yyin);
    yylex_destroy();
    if (status != 0 || lexical_error)
        return 1;
    puts("Syntax accepted.");
    return 0;
}
```

Control flow: open input, parse, close the file if needed, release scanner resources, then report the result. `yyerror()` suppresses a second diagnostic when scanner.l has already explained a lexical failure.

The supplied driver maps any nonzero parser result or `lexical_error` to process exit 1. Process exit 2 is reserved here for command-line usage and file-opening errors. These are the executable's exit codes, not a direct copy of every possible `yyparse()` return value.

Create build.sh and .gitignore

Location: quectoc-phase2-preview/build.sh. The script performs the same build as the original preview.

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")"
mkdir -p build
# -d produces the token header used by scanner.l.
bison -Wall -Werror=conflicts-sr -Werror=conflicts-rr -d \
    -o build/parser.tab.c parser.y
flex -o build/lex.yy.c scanner.l
gcc -std=c11 -Wall -Wextra -I build \
    build/parser.tab.c build/lex.yy.c main.c -o build/quectoc-parser
```

Instruction	Why it is present
set -euo pipefail	Stop the script when a build command fails; catch unset variables and failed pipelines.
cd to the script directory	Resolve scanner.l and parser.y relative to the project folder.
Bison before compilation	Produce parser.tab.c and parser.tab.h. The -d flag requests the header.
Conflict flags	Treat shift/reduce or reduce/reduce conflicts as build errors.
-I build	Tell GCC where to find the generated parser.tab.h header.
Explicit C filenames	Compile only the parser, scanner and driver for this executable.

Ignore generated files

Location: quectoc-phase2-preview/.gitignore. The leading dot is part of its name.

```
build/
build-cmake/
__pycache__/
```

Run **bash build.sh**; it does not require chmod +x. Flex's noyywrap option means this scanner does not require linking with -lfl. Keep files in Linux LF line-ending format when editing from Windows. [1]

Build and run checkpoint 1

```
bash build.sh
./build/quectoc-parser tests/one_declaration.qc
echo $?
```

Expected: Syntax accepted. followed by 0. Read the exit status immediately: echo \$? reports the status of the command that ran directly before it.

build/

```
├── parser.tab.c
├── parser.tab.h
├── lex.yy.c
└── quectoc-parser
```

generated by build.sh

Bison parser implementation

shared token definitions and yyparse

Flex scanner implementation

compiled executable

Compare a rejected input

```
./build/quectoc-parser tests/missing_semicolon.qc
echo $?
```

Expected: a syntax diagnostic and exit code 1. A diagnostic may vary slightly with the installed Bison version. A rejected input must never print Syntax accepted.

Use standard input

```
./build/quectoc-parser < tests/one_declaration.qc
```

The shell opens the file and redirects it to stdin. To type input interactively, run `./build/quectoc-parser` with no arguments, enter `let total=12;` and press Enter, then Ctrl+D on an empty terminal line. Enter alone does not end the input.

Rebuild after editing

After changing scanner.l, parser.y or main.c, run `bash build.sh` again before testing. Editing only a .qc input or tests/cases.json does not require recompilation. Never repair generated C files in build/: the next build replaces them.

Understand and run the tests

```
python3 tests/check.py build/quectoc-parser
```

Expected at each completed checkpoint: 28/28 checks passed. The suite runs 26 input files and adds checks for standard input and a missing source file. Intentional invalid programs count as passing tests when the executable rejects them correctly.

Path inside tests/	Role / action
*.qc	QuectoC input files. Keep these files when moving between checkpoints.
cases.json	Expected exit code per input; optional diagnostic text. Update it when the accepted grammar changes.
check.py	Runs the executable, checks exit codes, checks acceptance output, and checks requested diagnostic text.

Example entry in tests/cases.json

```
{
  "name": "missing_semicolon",
  "exit": 1,
  "diagnostic": "Syntax error"
}
```

This object refers to tests/missing_semicolon.qc. It is one entry in the complete JSON array; do not replace the entire manifest with this example.

Example input file	Checkpoint 1	Checkpoint 2	Checkpoint 3
one_declaration.qc	Accept	Accept	Accept
addition.qc	Reject	Accept	Accept
identifier_expression.qc	Reject	Accept	Accept
multiple.qc	Reject	Reject	Accept
print_unknown.qc	Reject	Reject	Accept
empty.qc / unary_minus.qc	Reject	Reject	Reject
unknown_character.qc	Lexical error	Lexical error	Lexical error

If a newly supported form is reported as a failing test, check that cases.json matches the checkpoint. Do not change an expected result merely to hide an implementation error.

Checkpoint 2 - add expressions

Edit: quectoc-phase2-preview/parser.y. Replace the grammar between its two %% markers with the following. Preserve the declarations above the first marker.

```
program:
    KW_LET IDENTIFIER ASSIGN expression SEMICOLON
;
expression:
    atom
    | expression PLUS atom
    | expression MINUS atom
;
atom:
    INT_LITERAL
    | IDENTIFIER
    | LPAREN expression RPAREN
;
```

An atom is an integer, an identifier or a parenthesized expression. Each recursive expression rule adds a binary + or - operation. The left-recursive form groups 12-3-1 as (12-3)-1; this checker does not calculate a result.

Update expectations, rebuild and test

```
cp Checkpoints/02_Expressions/tests/cases.json tests/cases.json
bash build.sh
./build/quectoc-parser tests/addition.qc
./build/quectoc-parser tests/parentheses.qc
./build/quectoc-parser tests/missing_operand.qc
echo $?
python3 tests/check.py build/quectoc-parser
```

addition.qc and parentheses.qc should be accepted. missing_operand.qc should give a syntax error and exit 1. The complete suite should report 28/28 checks passed.

Files changed: parser.y and tests/cases.json. scanner.l, main.c, build.sh and the .qc test inputs stay the same. This checkpoint still accepts only one declaration.

Reference comparison: diff -u Checkpoints/02_Expressions/parser.y parser.y. Comments or formatting may differ; compare the grammar productions.

Checkpoint 3 - add statements

Edit: quectoc-phase2-preview/parser.y. Replace its program rule with program, statements and statement below. Keep expression and atom. For clarity, the complete grammar between the %% markers is shown.

```
program:
    statements
;
statements:
    statement
    | statements statement
;
statement:
    KW_LET IDENTIFIER ASSIGN expression SEMICOLON
    | KW_PRINT LPAREN expression RPAREN SEMICOLON
;
expression:
    atom
    | expression PLUS atom
    | expression MINUS atom
;
atom:
    INT_LITERAL
    | IDENTIFIER
    | LPAREN expression RPAREN
;
```

```
cp Checkpoints/03_Statements/tests/cases.json tests/cases.json
bash build.sh
./build/quectoc-parser tests/multiple.qc
python3 tests/check.py build/quectoc-parser
```

Expected: Syntax accepted. and 28/28 checks passed. The nonempty statements rule allows one or more declarations or print statements. No value is printed by a QuectoC print statement in this preview.

Files changed: parser.y and tests/cases.json only. The full scanner still returns the same token kinds. print(unknown); is accepted syntactically because no declaration lookup is implemented.

File changes at a glance

Path from project root	Initial setup	Checkpoint 2	Checkpoint 3
scanner.l	Create / supplied	Keep	Keep
parser.y	One declaration	Replace grammar with expression and atom rules	Replace program; add statements and statement
main.c	Create / supplied	Keep	Keep
build.sh	Create / supplied	Run again	Run again
CMakeLists.txt	Supplied alternative	Keep; rebuild	Keep; rebuild
.gitignore	Create / supplied	Keep	Keep
tests/*.qc	Supplied inputs	Keep	Keep
tests/cases.json	Checkpoint 1 expectations	Copy checkpoint 2 expectations	Copy checkpoint 3 expectations
tests/check.py	Supplied runner	Run again	Run again
Reporting/notes.md	Template supplied	Record outcomes	Record outcomes
build/*	Generated	Regenerated	Regenerated
Checkpoints/*	Reference copies	Read / compare	Read / compare

Locate a file before editing it

```
pwd
ls scanner.l parser.y main.c build.sh
tree -a -L 2
```

In an editor, expand the root folder and open its parser.y. An edit under Checkpoints/02_Expressions/ does not alter the top-level executable. Every relative path in build.sh is resolved from the directory containing that script.

Keep a single entry point and one token vocabulary

main() belongs only in main.c. scanner.l and main.c include parser.tab.h from build/. Do not add a separate hand-written token-number header or compile every .c file recursively: the reference folders contain independent programs.

Add your evidence and save your work

Add one independent test

At checkpoint 3, create tests/my_program.qc with this content:

```
let base=10;
print((base+2)-1);
```

Run ./build/quectoc-parser tests/my_program.qc. Expect Syntax accepted. and exit 0. To include it in the suite, append the following object to the array in tests/cases.json, separating it from the previous object with a comma:

```
{"name": "my_program", "exit": 0}
```

The expanded suite should report 29/29 checks passed. Add a deliberate error next, predict whether it is lexical or syntactic, then run it and explain the result.

Complete Reporting/notes.md

Record your name, date, checkpoint and tool versions; one accepted input; one lexical failure; one syntax failure; your additional test; and the roles of yylex, yyparse and parser.tab.h. Explain why syntax acceptance does not prove that identifiers are declared.

Commit within the intended repository

If you created a new standalone local folder with no enclosing Git repository, run git init first. In Codespaces, use the repository already created by GitHub; avoid initializing another repository inside it.

```
git status
git add scanner.l parser.y main.c build.sh .gitignore
git add CMakeLists.txt tests Reporting README.md
git diff --cached
git commit -m "Build QuectoC Phase 2 standalone parser"
```

A commit records a local snapshot. If a GitHub remote is already configured, use your course submission workflow to push that commit. Stage the reference checkpoints and guide too if your submission requires the complete package.

Keep build/ and build-cmake/ excluded. Students should be able to recreate the executable from the committed source. Tool installations belong to the environment.

Optional CMake - the configuration

CMake automates the same generator and compiler steps. Use it from the terminal with the supplied top-level **CMakeLists.txt**. No IDE is required, and no scanner or grammar changes are needed.

```
cmake_minimum_required(VERSION 3.16)
project(QuectoCPhase2 C)
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
find_package(BISON 3.6 REQUIRED)
find_package(FLEX REQUIRED)
BISON_TARGET(QuectoCParser parser.y
  ${CMAKE_CURRENT_BINARY_DIR}/parser.tab.c
  DEFINES_FILE ${CMAKE_CURRENT_BINARY_DIR}/parser.tab.h
  COMPILE_FLAGS "-Wall -Werror=conflicts-sr -Werror=conflicts-rr")
FLEX_TARGET(QuectoCScanner scanner.l
  ${CMAKE_CURRENT_BINARY_DIR}/lex.yy.c)
ADD_FLEX_BISON_DEPENDENCY(QuectoCScanner QuectoCParser)
add_executable(quectoc-parser main.c
  ${BISON_QuectoCParser_OUTPUTS}
  ${FLEX_QuectoCScanner_OUTPUTS})
target_include_directories(quectoc-parser PRIVATE
  ${CMAKE_CURRENT_BINARY_DIR})
target_compile_options(quectoc-parser PRIVATE -Wall -Wextra)
```

Configuration	Purpose
find_package(...)	Locate installed Bison and Flex; require Bison 3.6 or newer.
BISON_TARGET / FLEX_TARGET	Describe how the .y and .l files produce generated C files. [4, 5]
ADD_FLEX_BISON_DEPENDENCY	Declare the scanner dependency on Bison-generated definitions. [5]
CMAKE_CURRENT_BINARY_DIR	Put generated files in the selected build directory, separate from source.
add_executable / include path	Compile the driver and generated sources; locate parser.tab.h.

The COMPILE_FLAGS spelling keeps this file compatible with CMake 3.x. CMake 4.x accepts it too; its newer OPTIONS spelling requires CMake 4.0 or later. [4]

Optional CMake - build and rebuild

From the same project root, install CMake 3.16 or newer and build into its own output directory:

```
sudo apt install -y cmake
cmake --version
cmake -S . -B build-cmake
cmake --build build-cmake
./build-cmake/quectoc-parser tests/one_declaration.qc
python3 tests/check.py build-cmake/quectoc-parser
```

-S . selects the current source directory. **-B build-cmake** selects a separate build directory. The first command configures and generates build rules; the second performs the actual build.

build-cmake/	generated; do not edit
— CMakeCache.txt	cached configuration and tool paths
— CMakeFiles/	generated rules and object files
— Makefile	default Unix Makefiles generator
— cmake_install.cmake	generated CMake support file
— parser.tab.c	generated Bison parser
— parser.tab.h	generated shared token header
— lex.yy.c	generated Flex scanner
— quectoc-parser	compiled executable

The Makefile shown is for the usual default Unix Makefiles generator on Ubuntu. If a different generator is selected, its build-system files may differ; `cmake --build` remains the common build command.

Continue through the checkpoints

Make the same `parser.y` and `tests/cases.json` changes on pages 11-12. Replace `bash build.sh` with `cmake --build build-cmake`, then run the tests against `build-cmake/quectoc-parser`. CMake regenerates the affected sources when their inputs change.

Choose the matching executable: `build.sh` produces `build/quectoc-parser`; CMake produces `build-cmake/quectoc-parser`. Building one does not refresh the executable in the other folder. Neither generated directory is included in the ZIP.

Troubleshooting by symptom

Symptom	Likely cause and correction
flex, bison or gcc: command not found	Install the missing tools in the same Linux environment where you are building. A Windows installation does not install tools inside Codespaces.
Bison version requirement fails	Run <code>bison --version</code> . Use Bison 3.6 or newer; the grammar declares this requirement.
scanner.l or parser.y cannot be found	Return to the project root. Check <code>pwd</code> and <code>ls</code> . Run <code>bash build.sh</code> there.
parser.tab.h: No such file or directory	Run the complete build script. Bison must generate the header, and GCC needs <code>-I build</code> .
Multiple definition of main	Keep <code>main()</code> only in <code>main.c</code> . Do not append a standalone scanner driver or compile reference projects together.
Undefined reference to <code>yylex</code> or <code>yyparse</code>	Compile and link <code>main.c</code> , <code>build/lex.yy.c</code> and <code>build/parser.tab.c</code> together using <code>build.sh</code> .
Undefined reference to <code>yywrap</code>	Check that <code>scanner.l</code> retains <code>%option noyywrap</code> ; regenerate and recompile.
The program waits after Enter	Input is still open. End interactive stdin with <code>Ctrl+D</code> on an empty line, or pass a source filename.
Expressions pass, but the test suite fails	Copy the matching checkpoint's <code>tests/cases.json</code> into the working <code>tests/</code> folder.
Changes have no effect	Save the edited source and rebuild. Check that you edited the top-level files and ran the matching binary.
Exit 2 / No such file or directory	The driver could not open the input or received too many arguments. Check the filename relative to your current directory.
Shell errors containing a carriage return	Save <code>build.sh</code> with LF line endings, then run it with <code>bash build.sh</code> .

Diagnostics are not always bugs. Missing semicolons, unknown characters, empty input, int declarations and unary minus are intentionally rejected by this preview.

Completion checks and references

Before finishing

Check	Evidence
Environment	Bison 3.6+; Flex, GCC and Python available in the Linux terminal.
Reproducible build	bash build.sh completes and creates build/quectoc-parser.
Checkpoint alignment	Grammar and tests/cases.json belong to the same checkpoint.
Positive and negative behavior	Valid full input is accepted; missing punctuation and trailing invalid input are rejected.
Regression checks	28/28 supplied checks pass, or 29/29 after the example student test.
Source ownership	Edit .l, .y and .c sources; leave generated build files out of Git.
Learning record	Reporting/notes.md explains token recognition, syntax checking and the later semantic phase.

Verified package

The standalone project and all three reference checkpoints were built with Flex 2.6.4, GNU Bison 3.8.2 and GCC 13.3.0; each passed its 28 supplied checks. The generated C files compiled without warnings. The optional CMake build was also checked. See Validation.txt.

Source alignment

Core scanner, parser, driver, build script and tests follow the supplied MountainC_Phase_2_Quectoc_Introductory_Preview_Pack.zip. This companion adds the standalone working root, setup explanations, project diagrams and reporting template. The three grammar checkpoints retain the original preview's scope.

Official documentation

[1] [Flex manual - Interfacing with Yacc](#): scanner/parser token interface. See also [the Flex manual](#) for options, patterns and generated scanners.

[2] [GNU Bison manual - Parser C-Language Interface](#): yyparse and the functions used by the generated parser.

[3] [GitHub Docs - Creating a codespace for a repository](#): repository and branch setup, then Code > Codespaces.

[4] [CMake - FindBISON](#). [5] [CMake - FindFLEX](#). Generator targets, options and the scanner/parser dependency.

Prepared 20 September 2026. Next learning step: attach semantic values and construct AST nodes while preserving the accepted syntax.